Guia Operativa
Pipeline Engine (S00 -> C12)

Catalaxia Finance -- S00-S04 . RATIO . C10 . C11 . C12

Derivada de codigo real, --help y ejecucion verificada. Sin comportamiento inventado.

Guía operativa — pipeline `engine` (S00→C12)

Derivada exclusivamente de: `--help` real de cada script, lectura de código (`scripts/engine/*.py` , `engine/*.py`), migraciones, y ejecución real de `S00` en esta sesión. Donde el código no responde una pregunta, se marca `NO DETERMINABLE` — no se completa con supuestos.

Todos los comandos se ejecutan **desde la raíz del repo**, contra `cataloxia_engine@127.0.0.1:5433/cataloxia` salvo que se indique lo contrario (`EXT-010` bloquea el puerto 5432 sin `ENGINE_ALLOW_PROD=1`).

ARQUITECTURA Y TOPOLOGIA DE RUNTIME (F12)

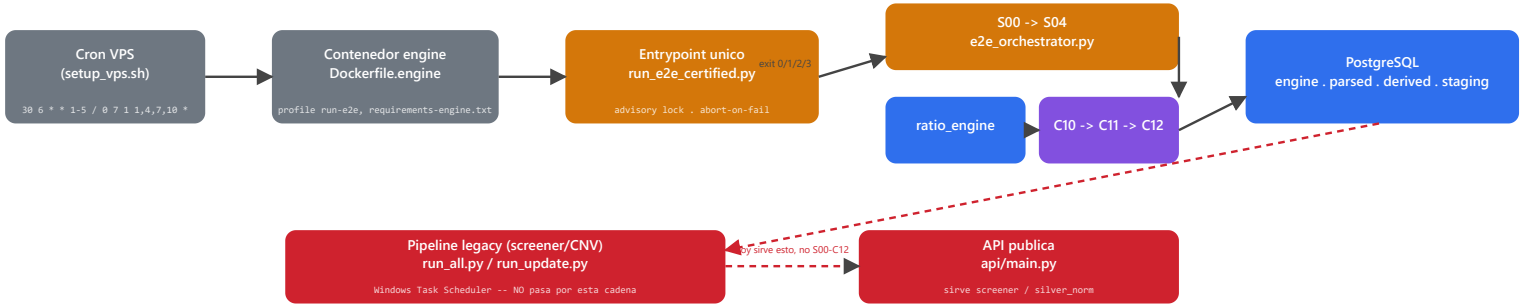


TABLA RESUMEN

Etapa	Script	Capa	Input principal	Output principal	Eje
S00	`scripts/engine/s00_smoke.py`	RAW (bootstrap)	ninguno (`checkpoint_id=ROOT`)	`engine.checkpoints` (S00), `engine.stage_runs`	ma `e2`
S01	`scripts/engine/s01_identity.py`	IDENTIDAD	`--cik` ; lee `identity.entities/identifiers/classifications`	`engine.checkpoints` (S01) — no escribe identidad, solo verifica	ma `e2`
S02	`scripts/engine/s02_fetch_ypf.py`	RAW	`--cik` , `--forms` , `--max-filings` ; red (SEC EDGAR)	archivos bajo `data/raw/engine` , `fetch_raw.download_log` , `engine.checkpoints` (S02)	ma `e2`
S03	`scripts/engine/s03_parse_ypf.py`	PARSED	`--cik` , `--min-xbrl` ; checkpoint S02 (mismo CIK)	`parsed.facts/contexts/units/concepts/link_relations/pre_xbrl_filings/companyfacts_compare/parse_errors` , `engine.checkpoints` (S03)	ma `e2`
S04	`scripts/engine/s04_normalize.py`	NORMALIZED	`--dry-run` ; checkpoint S03	`derived.fact` (upsert por `fact_key`), `derived.normalize_conflict` , `engine.checkpoints` (S04)	ma `e2`
RATIO	`scripts/engine/ratio_engine.py`	RATIOS	`--persist` , `--cik` ; `derived.fact`	`derived.ratio` (solo si `--persist`)	ma `ru`
C10	`scripts/engine/c10_evaluate.py`	VALIDACIÓN	`--sec-runs` , `--ciks` , `--tickers` , `--scope` ; lee `engine.*` / `derived.*`	JSON de veredicto (`--json-out`), solo lectura	ma `ru`
C11	`scripts/engine/c11_certificar.py`	CERTIFICACIÓN	`--run-id` , `--mode` , `--rule` ; lee `engine.runs/stage_runs/checkpoints`	`engine.certifications` (acta), JSON evidencia (`--json-output`)	ma `ru`
C12	`scripts/engine/publisher.py`	PUBLICACIÓN	`--source` , `--target` , `--run-id` , `--pk` ; exige C10 PASS + C11 CERTIFICADA	`staging.<target>__new__*` , `staging._publications` , swap atómico	ma `ru`

Auxiliares de orquestación (no son etapas propias, encadenan las de arriba):

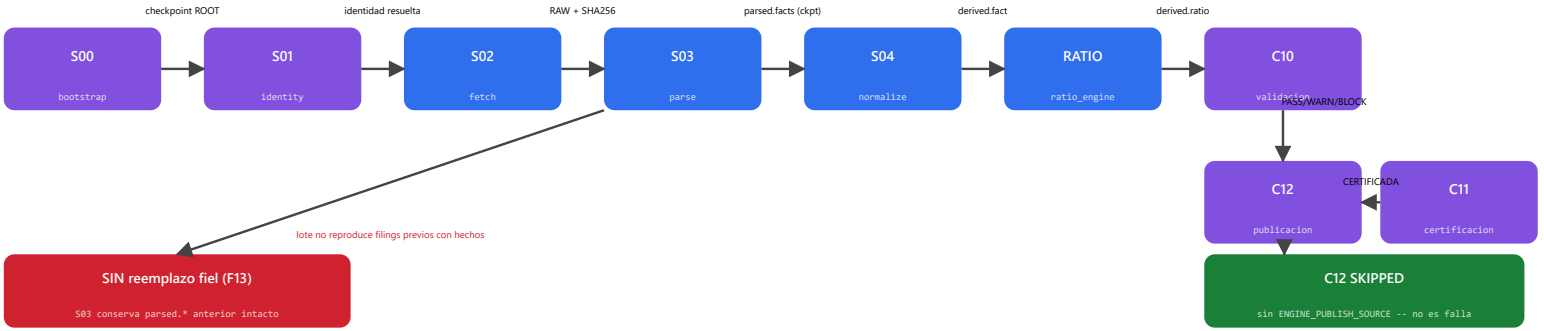
Script	Qué hace
`scripts/engine/e2e_orchestrator.py`	Encadena **S00→S04** como subprocesos bajo un mismo `run_id` (`ENGINE_E2E_RUN_ID`). No toca C10/C11/C12.
`scripts/engine/run_e2e_certified.py`	Entrypoint único de F12: encadena `e2e_orchestrator.py` (S00→S04) → `ratio_engine.py` → `c10_evaluate.py` → `c11_certificar.py` → `publisher.py`, con advisory lock y abort-on-fail.

MAPA DE EJECUCIÓN

```
S00 → S01 → S02 → S03 → S04 → RATIO → C10 → C11 → C12
```

- **S00 → S01**: S00 solo dejó constancia de que la base está sana (`engine.checkpoints` con `checkpoint_id=ROOT` como entrada). S01 **no lee ningún output de datos de S00** — verifica identidad independientemente (`identity.entities`). El encadenamiento real es de `run_id` / `stage_runs`, no de artefactos de datos.
- **S01 → S02**: S01 entrega la confirmación de que el CIK resuelve en `identity.entities` (`resolve_por_cik`); S02 la consume para saber a qué emisor pedirle filings a EDGAR.
- **S02 → S03**: S02 entrega archivos RAW en `data/raw/engine/` + filas en `fetch_raw.download_log` (con `sha256`). S03 los consume vía `find_s02_input_run()` — busca la última corrida S02 con gate PASS **del mismo CIK** y verifica integridad SHA256 antes de parsear (`C-S03-002`).
- **S03 → S04**: S03 entrega `parsed.facts` (+ contexts/units/concepts). S04 los consume y produce `derived.fact` con `source_fact_ids` → `parsed.facts.fact_id` (provenance).
- **S04 → RATIO**: `ratio_engine.py` lee `derived.fact` para calcular EV/Sales (único ratio verificado en el código leído; ver §RATIO).
- **RATIO → C10**: C10 lee `engine.stage_runs` / `engine.checkpoints` de los `run_ids` pasados por `--sec-runs`, y `derived.fact` para `NO_DANGLING`.
- **C10 → C11**: C11 lee el mismo `run_id` y certifica si `gate_status` de cada stage fue PASS/WARNING/BLOCK.
- **C11 → C12**: `publisher.py` exige, para publicar, que el `run_id` tenga certificación `CERTIFICADA` (`gate_decide`, verificado en auditorías anteriores de F10).

FLUJO DETALLADO DE ETAPAS -- QUE ENTREGA, QUE CONSUME



CAPAS Y PLANOS (harness arquitectónico)

Vista completa de las 12 capas del harness. Las que no tienen script propio (`FUENTES`, `ECONOMIC FACTS`, `FINANCIAL MODEL`, `PROVENANCE`) no aparecen en ningún encabezado de etapa más abajo — es la única sección donde quedan trazadas.

CAPA	PLANO primario	Componente	¿Tiene script propio?
FUENTES	DATOS	SEC EDGAR, CNV, yfinance, INDEC	no — externo
IDENTIDAD	CONTROL	S01_identity	sí
RAW	DATOS	S02_fetch	sí
PARSED	DATOS	S03_parse	sí
NORMALIZED	DATOS	S04_normalize	sí
ECONOMIC FACTS	DATOS	`econ_key` (dentro de S04, ver `06_PROVENANCE.md` §4)	no — campo, no etapa
FINANCIAL MODEL	CONTROL	`ratio_defs.py` (definiciones/fórmulas)	no — definiciones, no ejecución
RATIOS	DATOS	ratio_engine	sí
PROVENANCE	CONTROL	`source_fact_ids` / `fact_key` (mantenido por S03/S04/RATIO)	no — invariante transversal
VALIDACIÓN	CONTROL	C10	sí
CERTIFICACIÓN	CONTROL	C11	sí
PUBLICACIÓN	CONTROL + DATOS	C12	sí

El plano EJECUCIÓN (orquestación: `run_scope`, `e2e_orchestrator.py`, `run_e2e_certified.py`, `engine.stage_runs`) y el plano INFRAESTRUCTURA (`Dockerfile.engine`, `docker-compose.yml`, cron, migraciones, roles) son transversales a las 12 capas — no pertenecen a una sola fila de la tabla, las atraviesan todas.



S00_bootstrap

CAPA: RAW (bootstrap del ciclo de checkpoints) **PLANO:** EJECUCIÓN **FUNCIÓN:** Verifica que la migración `001_s00_bootstrap.sql` esté aplicada y que el destino no sea producción; deja el primer checkpoint (`checkpoint_id=ROOT`) del que cuelga toda la cadena.

INPUT

- Sin parámetros CLI (no tiene `argparse` ; no acepta `--help` como flag real — al ejecutarlo, corre igual).
- Variables de entorno: las que resuelve `engine.config.load_settings()` (puerto, credenciales, `ENGINE_MODE` , `ENGINE_ALLOW_PROD`).
- Dependencia: ninguna corrida previa (`input_checkpoint_id="ROOT"` , `hash_in=sha256(b"ROOT")`).

OUTPUT

- `engine.stage_runs` : transición `PLANNED→READY→RUNNING→VALIDATING→ …` .
- `engine.checkpoints` : nueva fila `STAGE_ID=S00_bootstrap` .
- Reporte JSON impreso por stdout (ver ejemplo real abajo).

COMANDO

```
python -m scripts.engine.s00_smoke
```

FLUJO

1. `load_settings()` — resuelve destino y aborta si es prod sin `ENGINE_ALLOW_PROD=1` (`EXT-010`).
2. Aplica migraciones pendientes si las hay.
3. Abre `run_scope( … , stage_id="S00_bootstrap")` .
4. Corre los checks `C-S00-001` / `C-S00-002` .
5. Escribe el checkpoint con `hash_in` / `hash_out` .
6. Imprime el `StageReport` JSON y el resumen de conciliación.

VALIDACIÓN

- **Gates**: `C-S00-001` (blocking) — `001_s00_bootstrap.sql` registrado en `engine.schema_migrations` . `C-S00-002` (blocking) — el target no es producción (puerto ≠ 5432, o `ENGINE_ALLOW_PROD=1` explícito).
- **PASS**: ambos checks `PASS` → `gate.status=PASS` .
- **FAIL/BLOCK**: cualquiera de los dos en `FAIL` → `gate.status` distinto de `PASS` (el código exacto de bloqueo no fue leído en detalle — para el valor textual exacto: `NO DETERMINABLE` sin más lectura).
- **Evidencia**: el JSON completo impreso por stdout (ver ejemplo real).

SIGUIENTE

S01_identity.

EJEMPLO REAL

Ejecutado en esta sesión, DEV real (`127.0.0.1:5433`):

```
$ python -m scripts.engine.s00_smoke
[config] target=catalaxia_engine@127.0.0.1:5433/catalaxia mode=FAST prod=False
[migrations] aplicadas ahora: (ninguna nueva)
[run] run_id=d3917984731a4939b746422bd5d6bcfd git=f375090326a2a8ca3091cb9157947d0235daea21
{"stage": "S00_bootstrap", "run_id": "d3917984731a4939b746422bd5d6bcfd", "mode": "FAST",
 "gate": {"status": "PASS", "reasons": [],
   "checks": [
     {"id": "C-S00-001", "result": "PASS", "blocking": true, "detail": "schema_migrations registra 001_s00_bootstrap.sql (1)"},
     {"id": "C-S00-002", "result": "PASS", "blocking": true, "detail": "target no es produccion (port=5433)"}
   ]}, ...}
--- conciliacion ---
rows_expected   = 2
rows_parsed     = 2
rows_rejected   = 0 {}
gate            = PASS
```

S01_identity

CAPA: IDENTIDAD **PLANO:** CONTROL **FUNCIÓN:** Verifica que el CIK objetivo resuelva de forma única y autoritativa en `identity.entities` antes de permitir el fetch; no escribe identidad, solo la confirma.

INPUT

- `--cik CIK` — CIK objetivo, 10 dígitos (default: golden case YPF, valor exacto no confirmado por `--help`; ver código para el default real — `NO DETERMINABLE` sin leer la línea de `add_argument`).
- Lee: `identity.entities`, `identity.identifiers`, `identity.identifier_reviews`, `identity.classifications` (vía `resolve_por_cik`).

OUTPUT

- `engine.checkpoints` (STAGE_ID= `S01_identity`).
- `engine.stage_runs` actualizado.
- No escribe ninguna tabla de `identity.*` (solo lectura).

COMANDO

```
python -m scripts.engine.s01_identity --cik 0000904851
```

FLUJO

1. Abre `run_scope(..., stage_id="S01_identity")`.
2. `resolve_por_cik(engine, cik)`.
3. Si el CIK no está presente o no coincide → `SystemExit("BLOCK: identidad inconsistente...")`.
4. Corre checks `C-S01-001..004`.
5. Escribe checkpoint con el resultado.

VALIDACIÓN

- **Gates**: `C-S01-001`, `C-S01-002`, `C-S01-003`, `C-S01-004` (contenido exacto de cada uno: `NO DETERMINABLE` sin leer el cuerpo completo del archivo más allá de lo relevado en esta sesión).
- **PASS/FAIL**: mismo patrón `gate.status` que S00.
- **Evidencia**: `StageReport` JSON por stdout + fila en `engine.checkpoints`.

SIGUIENTE

S02_fetch.

EJEMPLO REAL

`NO DETERMINABLE` — no se ejecutó en esta sesión (evitado para no generar corridas reales adicionales fuera de lo estrictamente necesario). El `--help` real:

```
usage: s01_identity.py [-h] [--cik CIK]
S01 identidad (por CIK; YPF=golden)
```

S02_fetch

CAPA: RAW **PLANO:** EJECUCIÓN **FUNCIÓN:** Descarga los filings del CIK desde SEC EDGAR y los deja en RAW inmutable con su hash, sin parsear.

INPUT

- `--cik CIK` — CIK objetivo, 10 dígitos.
- `--forms FORMS` — formularios anuales a ingerir (20-F extranjeros, 10-K domésticos).
- `--max-filings MAX_FILINGS` — tope de filings más recientes; `0` = todos.
- Dependencia: identidad resuelta (S01); red (SEC EDGAR).

OUTPUT

- Archivos bajo `data/raw/engine/` (`RAW_ROOT` , `engine/fetch.py:10`).
- `fetch_raw.download_log` (una fila por archivo: `raw_path` , `http_status` , `content_type` , `bytes` , `sha256` , `downloaded_at`).
- `engine.checkpoints` (STAGE_ID= `S02_fetch`).

COMANDO

```
python -m scripts.engine.s02_fetch_ypf --cik 0000904851 --forms 20-F --max-filings 0
```

FLUJO

1. Abre `run_scope( ... , stage_id="S02_fetch")` .
2. Resuelve identidad (mismo patrón que S01).
3. Descarga con `Fetcher` / `write_raw_new` — nunca sobre-escribe un archivo RAW existente (inmutabilidad).
4. Inserta cada descarga en `fetch_raw.download_log` con su `sha256` .
5. Corre checks `C-S02-001..005` .
6. Escribe checkpoint.

VALIDACIÓN

- **Gates**: `C-S02-001` a `C-S02-005` (contenido exacto de cada uno: `NO DETERMINABLE` sin lectura línea a línea adicional).
- **Evidencia**: `fetch_raw.download_log` + checkpoint JSON.

SIGUIENTE

S03_parse.

EJEMPLO REAL

`NO DETERMINABLE` — no ejecutado en esta sesión (requiere red real contra SEC EDGAR). `--help` real:

```
usage: s02_fetch_ypf.py [-h] [--cik CIK] [--forms FORMS] [--max-filings MAX_FILINGS]
```

S03_parse

CAPA: PARSED **PLANO:** DATOS + CONTROL **FUNCIÓN:** Parsea los filings XBRL descargados por S02 en hechos estructurados; reemplaza el `parsed.*` previo del CIK **solo si** el lote nuevo reproduce fielmente lo anterior (guarda F13, `decidir_reemplazo`).

INPUT

- `--cik CIK` — debe coincidir con el CIK del fetch de S02 que se va a parsear.
- `--min-xbrl MIN_XBRL` — mínimo de filings con XBRL parseado que exige `C-S03-004`.
- Dependencia obligatoria: última corrida S02 con `gate_status=PASS` del mismo CIK (`find_s02_input_run`); verifica integridad SHA256 contra `fetch_raw.download_log` antes de parsear (`C-S03-002`).

OUTPUT

- `parsed.facts`, `parsed.contexts`, `parsed.units`, `parsed.concepts`, `parsed.link_relations`, `parsed.pre_xbrl_filings`, `parsed.companyfacts_compare`, `parsed.parse_errors`.
- Artefactos: `outputs/docs/fase_s03/` (XLSX + MANIFEST + VALIDATION).
- `engine.checkpoints` (STAGE_ID= `S03_parse`) — **solo se actualiza si hubo reemplazo** (`reemplazo_ok=True`); si no, el checkpoint anterior sigue vigente.

COMANDO

```
python -m scripts.engine.s03_parse_ypf --cik 0000904851 --min-xbrl 9
```

FLUJO

1. Abre `run_scope( ... , stage_id="S03_parse")`.
2. Resuelve identidad, ubica el checkpoint S02 de entrada, verifica integridad RAW.
3. Parsea **todo en memoria** (sin escribir) — `parse_filing` por accession.
4. `decidir_reemplazo(previas_con_hechos, filing_results)`: solo si el lote nuevo reproduce todos los filings previos con hechos Y trae hechos nuevos.
5. Si `reemplazo_ok`: `DELETE` + `INSERT` del `parsed.*` del CIK, **en la misma transacción**, un solo `commit()`.
6. `C-S03-010` (bloqueante, primero de la lista) + `C-S03-001..009`.
7. Checkpoint solo si el gate dio PASS.

VALIDACIÓN

- **Gates bloqueantes**: `C-S03-001` a `C-S03-008`, y `C-S03-010` (guarda F13). `C-S03-009` es informativo (no bloqueante).
- **PASS**: `reemplazo_ok=True` (o sin necesidad de reemplazo) y el resto de los checks bloqueantes en PASS → `READY_FOR_S04`.
- **FAIL/BLOCK**: `reemplazo_ok=False` → `C-S03-010=FAIL` → `BLOCKED_S04`, el `parsed.*` anterior queda intacto, el checkpoint no se actualiza.
- **Evidencia**: `S03_VALIDATION.json` (incluye la clave `reemplazo`), `outputs/docs/fase_s03/*`.

SIGUIENTE

S04_normalize.

EJEMPLO REAL

De la certificación F13 (`docs/review/24_F13_IMPLEMENTATION_RESULT.md`, ejecución real verificada independientemente en auditoría previa): corrida `9d5cbc1a…` — `PASS 90.398 facts, reemplazo 17/17, hash_out eedb7af3…`.

S04_normalize

CAPA: NORMALIZED **PLANO:** DATOS **FUNCIÓN:** Normaliza `parsed.facts` a `derived.fact` con clave estable (`fact_key`), conservando provenance (`source_fact_ids`); nunca borra.

INPUT

- `--dry-run` — no escribe, solo corre el gate previo.
- Dependencia: checkpoint S03 vigente (`parsed.*` del CIK).

OUTPUT

- `derived.fact` — **upsert** por `fact_key` (`ON CONFLICT (fact_key) DO UPDATE SET ...` , línea 443).
- `derived.normalize_conflict` (conflictos de normalización).
- `engine.checkpoints` (STAGE_ID= `S04_normalize`).

COMANDO

```
python -m scripts.engine.s04_normalize
```

FLUJO

1. Abre `run_scope( ... , stage_id="S04_normalize")` .
2. Lee `parsed.facts` del checkpoint S03 vigente.
3. Calcula `fact_key = f(entity_key, accession, qname, context_id, unit_ref)` .
4. `INSERT ... ON CONFLICT (fact_key) DO UPDATE` — nunca `DELETE` .
5. Corre checks `C-S04-001..009` .
6. Escribe checkpoint.

VALIDACIÓN

- **Gates**: `C-S04-001` a `C-S04-009` (contenido exacto de cada uno: `NO DETERMINABLE` sin lectura adicional más allá de lo relevado).
- **Evidencia**: huella `md5` de `fact_keys` de `derived.fact` vigente (usada en la verificación de idempotencia de F13).

SIGUIENTE

ratio_engine (RATIO).

EJEMPLO REAL

De la certificación F13: corrida `9d5cbc1a…` — `PASS 45.113, checkpoint 9d5cbc1a10c54209-s04, hash_out eedb7af3…` .

RATIO (ratio_engine)

CAPA: RATIOS **PLANO:** DATOS **FUNCIÓN:** Motor canónico de ratios — calcula y opcionalmente persiste **cuatro** `canonical_id` : `eps` , `pb` , `pe` , `ev_sales` (`CALCULADORES = {"ev_sales": ..., "eps": ..., "pb": ..., "pe": ...}` , `calcular_todos()` los corre los cuatro — código verificado, no solo el `--help` , que en su docstring solo destaca EV/Sales).

INPUT

- `--persist` — escribe en `derived.ratio` (sin esta bandera, solo calcula).
- `--cik CIK` — un solo emisor; por defecto, todos.
- Dependencia: `derived.fact` (S04).

OUTPUT

- `derived.ratio` (solo si `--persist`).

COMANDO

```
python -m scripts.engine.ratio_engine --persist --cik 0000904851
```

FLUJO

1. Por cada `canonical_id` en `("eps", "pb", "pe", "ev_sales")`, corre su calculador (`calcular_eps`, `calcular_pb`, `calcular_pe`, `calcular_ev_sales`).
2. Cada calculador resuelve su propio contexto/insumos vía `_contexto()` / `insumo()`; si falta alguno, el resultado queda `NOT_CALCULABLE` con `motivo` (no aborta la corrida completa).
3. Sin `--persist`: solo imprime, no escribe.
4. Con `--persist`: upsert en `derived.ratio` versionado por `valid_to` (mismo patrón de S04).

VALIDACIÓN

No tiene un gate `C-RATIO-*` propio (no se encontró ningún check con ese prefijo en el archivo). El script no bloquea la cadena por sí mismo: en `run_e2e_certified.py` el paso `RATIO` se considera `ok` únicamente por `exit_code == 0` del proceso. La validación de cada ratio individual es a nivel de dato: `status = CALCULADO` o `NOT_CALCULABLE` con `motivo` en `derived.ratio`, nunca un valor aproximado. **Nota**: los 14 ratios contables de `ratio_defs.RATIOS` (current_ratio, roe, roa, etc. — ver `05_RATIO_REFERENCE.md`) **no los persiste esta etapa**; solo los cuatro de mercado/EPS de arriba llegan a `derived.ratio` (`SOFTWARE GAP SG-03`, documentado en `05_RATIO_REFERENCE.md`).

SIGUIENTE

C10 (validación).

EJEMPLO REAL

Del report real `outputs/e2e/e2e_2026-09-09T045350.json`: `{"step_id": "RATIO", "script": "ratio_engine.py", "exit_code": 0, "ok": true}`.

C10_evaluate

CAPA: VALIDACIÓN **PLANO:** CONTROL **FUNCIÓN:** Evaluador de invariantes de todo lo producido por S00-S04 (y opcionalmente Market/CNV), **solo lectura**; nunca escribe ni certifica.

INPUT

- `--sec-runs SEC_RUNS` — `run_id`s SEC separados por coma.
- `--ciks CIKS`, `--tickers TICKERS`.
- `--scope {sec,market,cross,cnv,all}` — determina qué invariantes corren. Para `scope=sec`: `SEC_GATES`, `IDENTITY`, `PROVENANCE`, `NO_DANGLING`, `NO_ORPHAN_ENTITY` (línea 199 del código; confirmado literal).
- `--json-out JSON_OUT`.

OUTPUT

- Archivo JSON en `--json-out` con `verdict` (`PASS` / `WARN` / `BLOCK`) y el detalle de cada invariante. **No escribe ninguna tabla.**

COMANDO

```
python -m scripts.engine.c10_evaluate --sec-runs 836dfbd566a440c3ba8eff392f6649f9 --ciks 0000904851 --tickers YPFD --scope sec --json-out outputs/e2e/c10.json
```

FLUJO

1. Recibe `run_ids` / `ciks` / `tickers` / `scope`.
2. Selecciona el set de invariantes según `scope` (`SCOPES` dict).
3. Corre cada invariante (`inv_sec_gates`, `inv_identity`, `inv_provenance`, `inv_no_dangling`, `inv_no_orphan_entity`, y para otros scopes: `inv_bridge`, `inv_separation`, `inv_no_cross_overwrite`, `inv_market_present`, `inv_cnv_scope`).
4. Agrega el veredicto final.
5. Escribe el JSON de salida.

VALIDACIÓN

- `NO_DANGLING`: `dangling=0` → PASS; `dangling>0` → BLOCK (query exacta: `derived.fact` con `source_fact_ids` que no resuelven en `parsed.facts`, filtrado por `valid_to IS NULL`).
- **Veredicto global**: `PASS` / `WARN` / `BLOCK` según el peor resultado de los invariantes evaluados.
- **Evidencia**: JSON completo con un objeto por invariante (`id`, `result`, `evidence`).

SIGUIENTE

C11_certificar.

EJEMPLO REAL

`outputs/e2e/c10_836dfbd566a440c3ba8eff392f6649f9.json` (verificado independientemente contra la base en auditoría previa):

```
{"verdict": "PASS",
 "invariants": [
   {"id": "SEC_GATES", "result": "PASS"},
   {"id": "IDENTITY", "result": "PASS"},
   {"id": "PROVENANCE", "result": "PASS"},
   {"id": "NO_DANGLING", "result": "PASS", "evidence": {"dangling": 0}},
   {"id": "NO_ORPHAN_ENTITY", "result": "PASS"}
 ]}
```

C11_certificar

CAPA: CERTIFICACIÓN **PLANO:** CONTROL **FUNCIÓN:** Emite un acta de certificación única y determinista por `run_id`, leyendo la evidencia ya producida (nunca calcula, nunca publica).

INPUT

- `--run-id RUN_ID` — run a certificar (default: último terminal).
- `--mode MODE` — filtra por `FAST` / `FULL` / `AUDIT`.
- `--rule RULE` — regla/versión de certificación (default `c11-rule-v1`).
- `--json-output JSON_OUTPUT`.
- Lee: `engine.runs`, `engine.stage_runs`, `engine.checkpoints`.

OUTPUT

- `engine.certifications` — **única escritura** (INV1, READ-ONLY sobre capas 1-10). Un `cert_id` por `run_id`, re-ejecución idempotente (`ON CONFLICT DO NOTHING`, nunca reescribe el acta — INV6).
- JSON de evidencia en `--json-output` (artefacto, no fuente primaria — INV5).

COMANDO

```
python -m scripts.engine.c11_certificar --run-id 836dfbd566a440c3ba8eff392f6649f9 --json-output outputs/e2e
```

FLUJO

1. Lee `engine.stage_runs` / `engine.checkpoints` del `run_id`.
2. Regla determinista (INV4): cualquier `gate_status=BLOCK` → `NEGADA`; `WARNING` → `CERTIFICADA_CON_OBSERVACIONES`; todo `PASS` → limpia.
3. Calcula `cert_hash` = sha256 sobre `run_id + pipeline_version + run_status + reports` (canonicalizado, sin timestamps — INV3).
4. `INSERT ... ON CONFLICT (run_id) DO NOTHING` en `engine.certifications`.
5. Imprime el acta JSON.

VALIDACIÓN

- **Veredicto**: `CERTIFICADA` / `CERTIFICADA_CON_OBSERVACIONES` / `NEGADA`.
- **INV7**: hoy es **informativa** — no bloquea la API ni la publicación por sí sola (eso queda del lado de `publisher.py` /C12, no de C11).
- **Evidencia**: acta completa (`stages_total` / `stages_pass` / `stages_warn` / `stages_block`, lista `evidence` por stage con `report_sha256`).

SIGUIENTE

publisher.py (C12).

EJEMPLO REAL

`outputs/e2e/certificacion_c11_836dfbd566a440c3ba8eff392f6649f9.json` (verificado independientemente contra `engine.certifications`):


```
{"certification_id": "c11-836dfbd566a440c3ba8eff392f6649f9",
 "run_id": "836dfbd566a440c3ba8eff392f6649f9",
 "veredicto": "CERTIFICADA",
 "cert_hash": "ef207c88828c1e835a7d6284ef53c4289aad51858f79b894815d5da1c620b629",
 "stages_total": 5, "stages_pass": 5, "stages_warn": 0, "stages_block": 0}
```


---

publisher (C12)

CAPA: PUBLICACIÓN **PLANO:** CONTROL + DATOS **FUNCIÓN:** Publica un snapshot a `staging.<target>` con swap atómico, sin cutover a producción; exige certificación previa.

INPUT

- `--source SOURCE` — `schema.tabla` origen (solo lectura).
- `--target TARGET` (**obligatorio**) — nombre lógico en staging.
- `--run-id RUN_ID` — run certificado (C10+C11).
- `--pk PK` — columnas PK separadas por coma.
- `--filter FILTER` — `WHERE` de lectura (solo `SELECT`).
- `--min-rows MIN_ROWS`.
- `--dry-run` — build+validate sin swap.
- `--rollback` — revertir al snapshot anterior.

OUTPUT

- `staging.<target>__new__<attempt>` (tabla de trabajo).
- `staging._publications` (bitácora: `run_id`, `input_hash`, `content_hash`, `c10_pass`, `certification_id`).
- Swap atómico a `staging.<target>` si todo PASS.

COMANDO

```
python -m scripts.engine.publisher --source derived.fact --target engine_facts --run-id 836dfbd566a440c3ba8eff392f6649f9 --pk fact_key --dry-run
```

FLUJO

1. Verifica `C10 PASS` + `C11 CERTIFICADA` para el `run_id`.
2. Lee `--source` con `--filter`, arma `staging.<target>__new__*`.
3. Calcula `content_hash` / `input_hash`.
4. Si `--dry-run`: valida y se detiene (`DRY_RUN_OK`).
5. Si no: swap atómico + registro en `staging._publications`.

VALIDACIÓN

- **PASS**: `res.ok=True`, `reason` en (`PUBLISHED`, `DRY_RUN_OK`).
- **BLOCK**: sin certificación → `BLOCK:C10_NO_PASS` / veredicto C11 distinto de `CERTIFICADA`; staging vacío → `BLOCK:staging vacio`.
- **Evidencia**: `PublishResult` (`ok`, `reason`, `target`, `rows`, `input_hash`, `content_hash`, `certification_id`, `cert_hash`).

SIGUIENTE

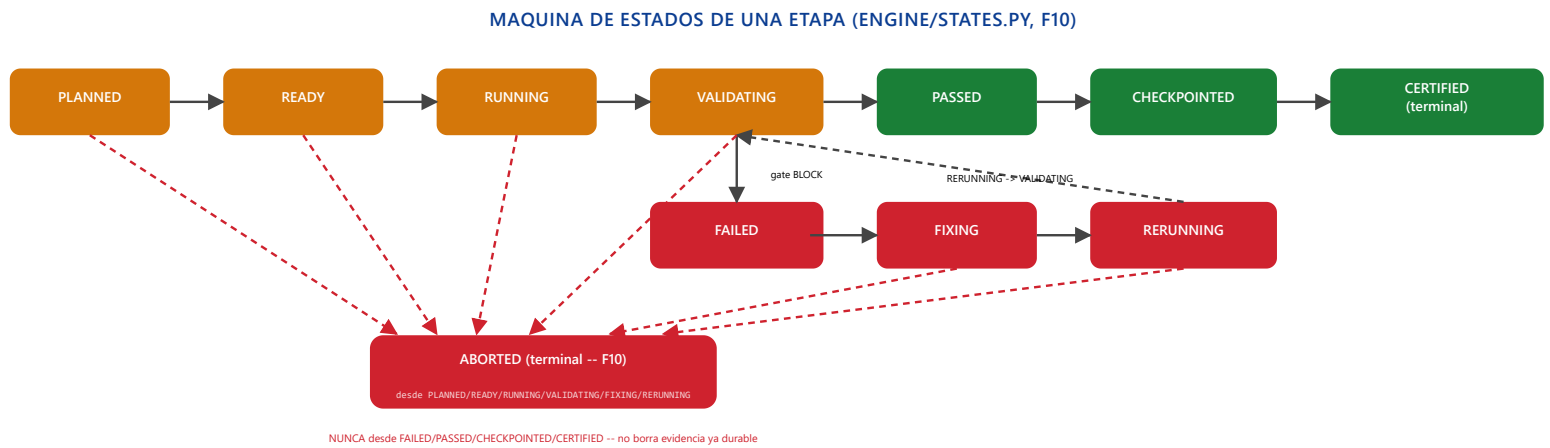
Fin de la cadena certificada (integración con el destino de negocio final: decisión diferida, ver `docs/review/26_F12_ARCHITECTURE_DECISION.md`).

EJEMPLO REAL

Del report real `e2e_2026-09-09T045350.json`, corrida sin `ENGINE_PUBLISH_SOURCE` declarado:

```
{"step_id": "C12", "script": "publisher.py", "exit_code": 0, "ok": true, "skipped": true}
```

(`publish_skipped: true` — comportamiento SKIPPED declarado explícitamente, `run_e2e_certified.py` líneas 366-375).



COMANDOS RÁPIDOS

```
# Ayuda de cada script (real, ejecutable)
python -m scripts.engine.s01_identity --help
python -m scripts.engine.s02_fetch_ypf --help
python -m scripts.engine.s03_parse_ypf --help
python -m scripts.engine.s04_normalize --help
python -m scripts.engine.ratio_engine --help
python -m scripts.engine.c10_evaluate --help
python -m scripts.engine.c11_certificar --help
python -m scripts.engine.publisher --help
python -m scripts.engine.e2e_orchestrator --help
python -m scripts.engine.run_e2e_certified --help

# Verificar destino antes de correr cualquier E2E (obligatorio)
python -c "from engine.config import load_settings; s=load_settings(); print(s.describe())"

# S00 solo (bootstrap/smoke)
python -m scripts.engine.s00_smoke

# Cadena completa S00→S04 bajo un mismo run_id
python -m scripts.engine.e2e_orchestrator --mode FAST --cik 0000904851

# Cadena certificada completa (S00→S04→ratio_engine→C10→C11→C12)
python -m scripts.engine.run_e2e_certified --mode FAST --cik 0000904851 --tickers YPFD

# Cadena certificada, publicando (requiere origen de negocio definido)
python -m scripts.engine.run_e2e_certified --mode FAST --cik 0000904851 ^
  --publish-source derived.fact --publish-target engine_facts --publish-pk fact_key

# Contenedor (F12): la misma cadena, dentro de Docker
docker compose --profile run-e2e run --rm engine
docker compose --profile run-e2e run --rm engine --mode FULL
```